



epsi®

l'école d'ingénierie
informatique



wis®

l'école supérieure
des métiers du digital

RAPPORT DE STRATÉGIE DE MIGRATION

Plateforme MedAssist

Année 2025-2026 Bachelor SYSOPS

Réalisé par :

MAAS Romane, CORREIA DA SILVA Rafael, TARASSE Saossane, KABA Tasilimy

Sommaire

1. Introduction et objectifs	2
2. Synthèse des contraintes et exigences	2
2.1. Contraintes de service et réglementaires	2
2.2. Contraintes techniques de migration	2
3. Analyse détaillée des évolutions (A à E)	3
Évolution A – Restructuration de l'adresse des patients	3
Évolution B – Normalisation du champ doctor_name	3
Évolution C – Refonte du champ gender dans patients	3
Évolution D – Chiffrement du SSN (données sensibles)	3
Évolution E – Partitionnement de la table consultations	3
3.1. Justification du rejet des stratégies alternatives	4
4. Plan de séquençement global	4
Phase 1 : Préparation et extension (Expand)	4
Phase 2 : Synchronisation et migration de fond (Backfill)	4
Phase 3 : Bascule structurelle (Maintenance Window)	5
Phase 4 : Nettoyage (Contract)	5
5. Analyse d'impact et gestion des risques (C1-C6)	6
6. Stratégie de rollback	7
7. Plan de validation et résultats techniques	7
7.1. Critères de validation (Tests A à E)	7
7.2. Plan de monitoring	8
7.3. Exécution et résultats de la vérification technique	8

1. Introduction et objectifs

MedAssist est une plateforme SaaS de gestion de dossiers médicaux en production depuis 4 ans, utilisée par **350 cabinets médicaux** en France. Elle assure la gestion des dossiers patients, des consultations, des prescriptions et de la facturation.

L'architecture actuelle repose sur une API REST (Spring Boot) connectée à une base PostgreSQL 16. Le projet de migration vers la version 2.0 s'inscrit dans un contexte de forte croissance et d'exigences réglementaires accrues.

Chiffres clés de l'infrastructure :

- **Patients en base** : ~2,4 millions.
- **Consultations en base** : ~18 millions (croissance de ~12 000/jour).
- **Prescriptions en base** : ~45 millions.
- **Pic d'utilisateurs simultanés** : ~800.

2. Synthèse des contraintes et exigences

La stratégie de migration doit impérativement respecter les contraintes métier et techniques suivantes :

2.1. Contraintes de service et réglementaires

- **Zéro perte de données (C1)** : Les données de santé sont soumises aux réglementations **HDS** (Hébergement de Données de Santé) et au **RGPD**. Toute perte ou incohérence n'est pas envisageable .
- **Disponibilité maximale (C2)** : Le SLA contractuel est de **99,9%**, ce qui limite l'indisponibilité totale à **8h44 par an**.
- **Fenêtre de maintenance** : Les interventions lourdes ne sont autorisées que le **dimanche de 2h à 6h** (4 heures), période durant laquelle le service peut être dégradé.

2.2. Contraintes techniques de migration

- **Compatibilité V1/V2 (C4)** : Le déploiement s'effectue par *Rolling Update* via Kubernetes. Les instances applicatives V1 et V2 coexistent et doivent pouvoir fonctionner simultanément sur le même schéma de base de données pendant la transition.
- **Réversibilité (C3)** : Un mécanisme de **rollback** doit être possible à chaque étape du processus pour revenir à l'état précédent sans perte de données en cas d'échec.
- **Intégrité et Performance (C5, C6)** : L'intégrité référentielle doit être garantie en permanence. De plus, aucune dégradation des performances supérieure à 10% n'est tolérée (temps de réponse moyen actuel : 120ms).

3. Analyse détaillée des évolutions (A à E)

Évolution A – Restructuration de l'adresse des patients

- **Problème** : L'adresse est actuellement "en dur" dans la table patients, ce qui empêche d'avoir plusieurs adresses ou un historique.
- **Risques** : Le verrouillage de la table patients (~2,4 millions de lignes) lors de l'extraction des données. Une rupture de compatibilité si la V1 cherche l'adresse là où elle n'est plus.
- **Stratégie choisie** : **Expand-Contract (Parallel Change)**.
- Justification : Cette approche permet de créer la table addresses (Expand), de synchroniser les deux structures via des triggers le temps que les instances V1 et V2 coexistent (C4), puis de supprimer les anciennes colonnes (Contract).

Évolution B – Normalisation du champ doctor_name

- **Problème** : Le nom du médecin est un texte libre avec de nombreuses incohérences (ex: "Dr Martin" vs "Martin Jean").
- **Risques** : Création de doublons dans la nouvelle table doctors ou mauvais rattachement des consultations (18 millions de lignes).
- **Stratégie choisie** : **Expand-Contract avec Backfill par lots (Batching)**.
- Justification : Le volume de consultations (18M) rend un UPDATE massif impossible sans bloquer la base. Le nettoyage doit se faire progressivement pour isoler les cas d'incohérences sans interrompre le service.

Évolution C – Refonte du champ gender dans patients

- **Problème** : Passer d'un CHAR(1) ('M', 'F') à un format plus large ('NB', 'U') avec une table de référence.
- **Risques** : L'ancienne application (V1) pourrait planter si elle récupère une valeur comme 'NB' qu'elle ne sait pas traiter.
- **Stratégie choisie** : **Expand-Contract (Ajout d'une nouvelle colonne)**.
- Justification : On ajoute une nouvelle colonne gender_id (FK). Les triggers assurent que si la V1 écrit 'M', la V2 voit l'ID correspondant. C'est la seule façon de garantir la contrainte C4 (coexistence V1/V2).

Évolution D – Chiffrement du SSN (données sensibles)

- **Problème** : Le numéro de Sécurité Sociale doit être chiffré au repos via pgcrypto.
- **Risques** : Surcharge CPU lors du chiffrement de 2,4M de lignes et dégradation des performances lors des lectures (limite de 10% de perte de performance).
- **Stratégie choisie** : **Migration progressive par lots (Backfill batching)**.
- Justification : On crée une colonne ssn_encrypted. On chiffre les données par paquets de 10 000 lignes en arrière-plan pour ne pas saturer le serveur et respecter le SLA.

Évolution E – Partitionnement de la table consultations

- **Problème** : La table consultations est trop grosse (18M de lignes), ce qui ralentit les rapports.

- **Risques** : PostgreSQL ne permet pas de convertir une table existante en table partitionnée via ALTER TABLE. Il faut migrer 18 millions de lignes vers une nouvelle structure.
- **Stratégie choisie** : **Shadow Table + Rename/Swap**.
- **Justification** : On crée une table "ombre" déjà partitionnée, on y copie les données, puis on échange les noms (Swap) pendant la fenêtre de maintenance du dimanche (4h). C'est la méthode la plus appropriée pour une restructuration aussi lourde.

3.1. Justification du rejet des stratégies alternatives

Pour chaque évolution, nous avons considéré les approches suivantes mais les avons jugées sous-optimales :

- **Stratégie "Big Bang" (Arrêt total)** : Rejetée systématiquement pour les évolutions A, B, C et D. Bien que simple, elle impose une interruption totale du service qui risquerait de dépasser la fenêtre de 4h le dimanche. Elle ne permet pas non plus le *rolling deploy* exigé (C4).
- **Stratégie "Blue-Green Database"** : Rejetée en raison du volume de données (18M de consultations). Maintenir deux instances de production synchronisées doublerait les coûts d'infrastructure et complexifierait énormément la gestion de la cohérence des écritures.
- **Vues de compatibilité** : Bien qu'utiles pour la lecture, elles ont été écartées pour les phases d'écriture (INSERT/UPDATE). La gestion des triggers **INSTEAD OF** sur des tables complexes peut entraîner une dégradation des performances supérieure aux 10% autorisés (C6).

4. Plan de séquençement global

Pour respecter la contrainte de **coexistence V1/V2 (C4)** et le **SLA de 99,9%**, la migration ne sera pas effectuée en un seul bloc, mais découpée en quatre phases distinctes.

Phase 1 : Préparation et extension (Expand)

L'objectif est d'ajouter les nouvelles structures sans modifier les données existantes pour ne pas impacter la V1.

- **Actions** :
 - o Création de l'extension pgcrypto.
 - o Création des nouvelles tables : addresses, doctors, gender_ref.
 - o Ajout des colonnes temporaires : patients.ssn_encrypted, patients.gender_id et consultations.doctor_id.

Phase 2 : Synchronisation et migration de fond (Backfill)

Cette phase peut durer plusieurs jours et s'exécute en arrière-plan pendant que l'application tourne.

- **Actions :**

- **Mise en place des Triggers :** Tout insert/update sur la V1 alimente automatiquement les nouvelles structures (et inversement).
- **Backfill par lots :** Migration des 2,4 millions de SSN (chiffrement) et des adresses par paquets de 10 000 lignes pour éviter de saturer les entrées/sorties (I/O).
- **Normalisation des médecins :** Script de nettoyage pour rattacher les 18 millions de consultations aux 120 médecins identifiés.

Phase 3 : Bascule structurelle (Maintenance Window)

Utilisation de la fenêtre de maintenance du dimanche (2h-6h) pour les opérations les plus lourdes.

- **Actions :**

- Partitionnement (Évolution E) : Création de la table consultations_partitioned.
- Swap : Renommage de la table originale en consultations_old et de la nouvelle table en consultations.
- Validation : Tests d'intégrité et de performance immédiats.

Phase 4 : Nettoyage (Contract)

Une fois que 100% des instances applicatives sont passées en version 2.0.

- **Actions :**

- Suppression des colonnes obsolètes dans patients (ssn, address_*, gender).
- Suppression de la colonne doctor_name dans consultations.
- Suppression des triggers de synchronisation devenus inutiles.

Évolution	Type de migration	Impact service	Justification
A, B, C, D	En ligne (Expand-Contract)	Zéro downtime	Coexistence V1/V2 gérée par triggers.
E (Partition)	Shadow Table + Swap	Bref arrêt (< 15 min)	Nécessaire pour le renommage final de la table.

5. Analyse d'impact et gestion des risques (C1-C6)

- Voici un tableau récapitulatif de l'impact de chaque migration sur l'exploitation :

Évolution	Type d'impact	Durée estimée	Mesure d'atténuation des risques
A (Adresses)	Transparent	~2h (Backfill)	Exécution par lots de 5 000 lignes pour éviter le verrouillage de la table patients.
B (Médecins)	Transparent	~4h (Nettoyage)	Utilisation de fonctions de correspondance textuelle (levenshtein ou similarity) pour isoler les cas douteux avant validation.
C (Gender)	Transparent	< 30 min	Simple ajout de colonne et mise à jour de la table de référence.
D (SSN)	Charge CPU	~3h (Chiffrement)	Chiffrement progressif en heures creuses. Surveillance étroite du CPU pour ne pas impacter les requêtes API (C6).
E (Partition)	Bref arrêt	~1h (Swap final)	Réalisé exclusivement durant la fenêtre de maintenance du dimanche (2h-6h).

6. Stratégie de rollback

Conformément à la contrainte C3, chaque étape Flyway sera accompagnée d'un script de rollback (undo) permettant de :

1. Supprimer les nouveaux triggers de synchronisation.
2. Rétablir les contraintes de clés étrangères initiales.
3. Supprimer les colonnes et tables ajoutées en phase "Expand".
4. Note : Pour le chiffrement (D) et le partitionnement (E), le rollback s'appuie sur la conservation temporaire des données originales (colonne ssn en clair et table consultations_old) jusqu'à validation complète de la V2.

7. Plan de validation et résultats techniques

7.1. Critères de validation (Tests A à E)

La séquence de validation est critique et doit être exécutée dans l'ordre suivant pour garantir l'intégrité des structures et la compatibilité des versions. Le succès de ces tests constitue les critères d'acceptation finaux (OK/NOT OK) pour le passage en Phase 4 (Nettoyage).

Tests post-migration V2 (Tests A à E) :

- Test A (Adresses) : Vérifier l'intégrité des 2,4 millions d'adresses migrées de la colonne patients.address_ vers la nouvelle table addresses.
- Test D (Numéro de Sécurité Sociale - NSS) : S'assurer que le champ ssn_encrypted est chiffré et lisible par la Version 2 (V2). Confirmer également que l'ancienne colonne ssn n'est plus utilisée (lue ou mise à jour) par l'application.
- Test E (Partitionnement) : Mesurer les temps de réponse des requêtes select sur la table consultations après partitionnement. L'objectif est de confirmer que la dégradation des performances est maintenue sous le seuil de 10 % (objectif C6).

Test de Contract (Post-Phase 4) :

- Script `test\_contract.sql` : Exécuté après la suppression des colonnes obsolètes (Phase 4), ce script vérifie qu'aucune rupture de service ni erreur d'exécution n'est générée côté V2, assurant ainsi la cohérence du contrat final entre l'application et la base de données.
- Point d'attention critique : L'ordre d'exécution des tests V2 (A → E) doit être respecté avant le lancement du test CONTRACT. Un lancement prématuré du test CONTRACT pourrait générer de faux échecs.

7.2. Plan de monitoring

Un plan de monitoring précis est essentiel pour surveiller l'impact de la migration progressive par lots (Backfill) et l'exécution de la bascule structurelle.

Phase	Indicateur Clé	Objectif	Contrainte Adressée
Phase 2 (Backfill)-	Achèvement de la migration des données	Migration de 100% des 2,4 millions de SSN et 18 millions de consultations	C1 (Absence de perte de données)
Phase 2 (Backfill)-	Performance des API	Temps de réponse moyen inférieur à 132 ms (120 ms + 10%)	C6 (Performance)
Phase 2 (Backfill)-	Fiabilité des Triggers	Zéro erreur de synchronisation entre les versions V1 et V2	C5 (Intégrité des données)
Phase 3 (Swap) -	Durée de l'opération de bascule (RENAME/SWAP)	Moins de 15 minutes	Respect de la fenêtre de maintenance (4 heures)

7.3. Exécution et résultats de la vérification technique

La vérification technique de la stratégie a été effectuée sur un environnement de pré-production vierge pour garantir la validité de l'ensemble du cycle de vie (Flyway) et de la séquence de tests.

Environnement de test :

Un conteneur Docker avec une base de données PostgreSQL vierge a été utilisé pour valider le script d'installation Flyway complet (versions V1 à V4).

Procédure de vérification :

La procédure a été exécutée séquentiellement :

1. Réinitialisation complète du volume Docker.
2. Migration vers la version V2 (Phase 1).
3. Exécution des tests unitaires A, B, C, D et E.

4. Migration complète jusqu'à la version V4 (Phase 3).
5. Exécution du script test_contract.sql.

Résultats et conclusion

Les résultats des tests sont conformes :

- Tests EXPAND (V2) : **OK**
- Tests CONTRACT (V4) : **OK**
- Migration complète (V1 → V4) : **OK**